

gBPF: An Extensible OS Policy Interface for GPU Systems

Abstract

Performance in modern GPU-centric systems increasingly depends on resource management policies, such as memory placement, scheduling, and observability, rather than raw compute alone. Existing approaches force developers into a programmability and low-level control tradeoff: user-space runtimes offer programmability but lack cross-tenant visibility and fine-grained hardware control, while kernel-level approaches provide control but embed policy logic into privileged drivers, complicating maintenance and raising safety risks. We argue that the GPU driver and device layer must serve as an extensible OS policy interface. Recently emerging eBPF offers a possibility, but naively transplanting host-side eBPF is insufficient: it cannot observe critical device-side events, and directly injecting policy code into GPU kernels affects safety and efficiency.

We present gBPF, an eBPF-based policy runtime that treats the GPU driver and device as a programmable OS subsystem. gBPF extends GPU drivers to expose safe hooks and introduces a device-side eBPF runtime that executes verified policy logic within GPU kernels, enabling coherent, application-transparent policies. Evaluation on realistic workloads, including inference, training, and vector search, shows that gBPF improves throughput by up to $4.8\times$ and reduces tail latency by up to $2\times$ with low overhead, without modifying applications or restarting drivers.

1 Introduction

The performance and efficiency of GPU-based systems increasingly depend not only on raw compute and memory performance but also on implemented resource management policies. GPUs account for a growing share of compute resources in modern systems [24], supporting diverse workloads such as latency-sensitive inference [2, 28, 60], compute-intensive training [46, 50], graph analytics [8, 30, 56], vector search [5, 43], and data visualization [36], each imposing distinct and often conflicting resource requirements. GPU software stacks,

spanning user-space frameworks, OS kernel drivers, vendor firmware, and device-side kernel code, necessitate coordination of memory placement and migration [44, 49, 52, 53, 63], kernel scheduling in multi-tenant environments [15, 21, 23, 29, 37, 51, 58], and dynamic observability for online adaptation [1]. These policies must evolve alongside changing hardware architectures, emerging workloads, and varying service-level agreements (SLAs) [12, 13, 28, 34, 45, 61].

Unfortunately, existing GPU resource management solutions force developers into tradeoffs between user-space methods that lack global visibility and control over system-level GPU behaviors, and kernel-level implementations that introduce complexity and safety risks. Concretely, user-space frameworks [7, 17, 38, 48] offer programmability, but lack cross-application visibility and management of low-level GPU mechanisms (e.g. thread timeslices and replayable page faults [14, 39]). These approaches bind to specific runtime frameworks, requiring application-level code changes and limiting portability and reuse. Conversely, kernel-based resource management approaches [14, 26, 27, 57, 62] provide visibility and control of GPU hardware states, their direct modification on static, vendor-specific kernel modules making them difficult to deploy and maintain, and posing stability concerns. GPU profiling frameworks [22, 40, 54, 59] enable kernel-level observation but lack runtime programmability and driver integration, limiting adaptive resource management.

To address this, we argue that GPU resource management requires **an extensible OS policy interface for GPU** providing safe, flexible and transparent programmability. The GPU driver layer is positioned to offer global visibility, enable cross-tenant coordination, and provide fine-grained management of privileged low-level GPU mechanisms without requiring application modifications. Recent CPU-side frameworks, such as eBPF, have demonstrated the feasibility of dynamic, kernel-level policy programmability with safety guarantees [4, 25, 35, 64, 65]. Inspired by its success, we propose treating GPU resource management as a first-class programmable OS subsystem. However, naively extending host-only eBPF remains insufficient because scheduling and

memory management decisions such as thread-block dispatch, warp-level execution scheduling, and memory synchronization occur within GPU hardware and remain invisible and uncontrollable from the host. Thus, a practical solution must expose programmable policy hooks within GPU drivers and execute verified policy logic inside GPU kernels.

We present gBPF, a cross-layer policy runtime that transforms the GPU driver and device into a programmable OS subsystem. gBPF exposes stable, verified control-plane hooks within the GPU driver, enabling safe and customized programmable resource management policies. gBPF further supports injecting verified eBPF policy logic into GPU kernels, providing consistent and transparent policy enforcement across the host-device boundary.

Designing gBPF has three technical challenges to solve: **C1:** Exposing low-level driver mechanisms provides expressiveness but risks instability; we balance this tension with a narrow policy interface built around hardware-aligned abstractions (§4.3). **C2:** Scalar eBPF semantics conflict with GPU SIMT parallelism, causing divergence and deadlocks. Thus, we introduce a SIMT-aware verifier and warp-level execution model for optimization (§4.4). **C3:** Host-device memory hierarchies differ in latency and consistency; we bridge them with cross-layer eBPF maps using relaxed consistency (§4.4).

We implement gBPF on Linux by extending the NVIDIA open GPU kernel modules [39] and providing an eBPF runtime for GPU devices. We evaluate gBPF across GPU workloads including LLM inference pipelines, GNN training, vector search, and mixed-priority multi-tenant scenarios. Using gBPF, we implement adaptive memory prefetching and eviction policies, fine-grained kernel preemption strategies, dynamic work-stealing schedulers, and programmable GPU kernel observability tools. gBPF-based policies improve throughput by up to $4.8\times$ and reduce tail latency by up to $2\times$ compared to static heuristics, while instrumentation-only deployments incur low overhead. This demonstrates that cross-layer eBPF-based programmability spanning host and device is an effective abstraction for GPU-centric systems. In summary, our contributions are:

- We design an extensible OS policy interface for GPU memory, scheduling, and observability.
- We implement gBPF, an eBPF runtime with SIMT-aware verification spanning host driver and GPU device.
- We show gBPF policies improve throughput and reduce tail latency across inference, training, and search workloads.

2 Background and Motivation

2.1 The Diversity of GPU-Centric Workloads

Modern GPU systems run diverse workloads with conflicting resource demands. Generative AI inference exhibits compute-intensive prefill, bandwidth-bound decoding [44, 63], irreg-

ular KV-cache offloading [28, 49], and sparse expert activations [16, 20]. Dense neural network training demands predictable synchronization, while GNN and embedding workloads exhibit irregular, pointer-chasing accesses resistant to prefetching. Vector search adds variability with random-access query and sequential indexing. These distinct memory behaviors (Figure 1) and imbalanced thread scheduling across SMs (Figure 2) are poorly served by static policies. In multi-tenant environments, latency-critical inference conflicts with throughput-maximizing training, creating contention that fixed partitioning cannot resolve.

2.2 GPU Systems Architecture

Modern GPU software stacks consist of three major layers:

User-Space Logic (Applications & Runtimes) High-level frameworks (e.g., PyTorch) and runtimes (e.g., CUDA libraries) operate in user space, managing operation graphs, tensor layouts, and command submissions via memory-mapped interfaces or ioctl calls to drivers. These components encode rich semantic information, such as neural network structures and latency constraints. Runtime systems like vLLM [28], fine-grained sharing frameworks such as Salus [61], and UVM-focused runtimes like PILOT [47] implement resource management policies at the application level.

The One-size-fits-all Kernel Driver The GPU driver acts as the OS component for GPU systems, managing privileged low-level hardware mechanisms including the Memory Management Unit (MMU), hardware interrupts, and kernel scheduling. Typical drivers are monolithic, implementing application-agnostic resource-management algorithms such as Least Recently Used (LRU)-based page eviction, round-robin scheduling for context switching, and tree-based prefetching in Unified Virtual Memory scenarios. These drivers traditionally offer coarse-grained policies, intentionally abstracting away application-specific semantics.

Device-Side Execution The GPU device executes computations via user-defined device kernels and proprietary vendor firmware, which are compiled into static binaries specifying precise thread-level SIMT behaviors. GPU hardware schedulers manage thread execution at warp granularity. Device kernels and firmware lack runtime adaptability, making critical internal states (such as warp divergence, cache thrashing, and thread-level execution bottlenecks) opaque to host-side drivers. This architectural opacity constrains runtime workload-adaptive optimizations.

In summary, modern GPU software stacks include statically-defined policy logic embedded within monolithic kernel drivers, closed-source firmware, and opaque device

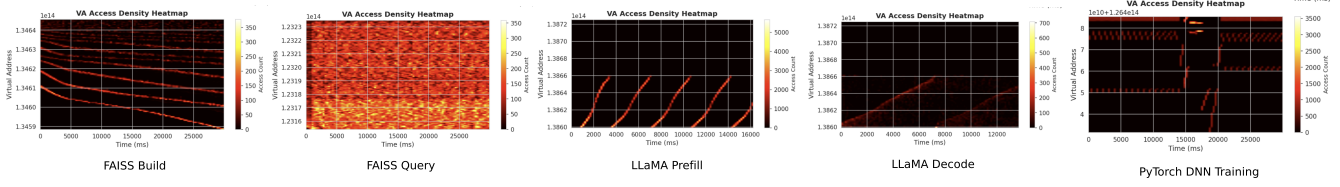


Figure 1: Memory access (page fault) patterns vary across GPU workloads: faiss Build exhibits sequential scans; faiss Query random accesses; llama.cpp MoE Prefill shows periodic sequential patterns while Decode exhibits sparse random accesses; PyTorch DNN shows periodic block accesses.

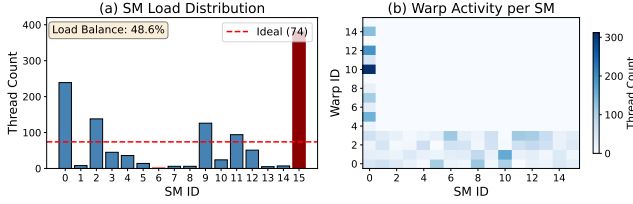


Figure 2: GPU thread scheduling imbalance observed via eBPF tracing. (a) SM load distribution shows $127\times$ imbalance (SM 15: 382 threads vs SM 6: 3 threads). (b) Warp activity heatmap reveals SM 0 concentrates work in high-numbered warps while other SMs underutilize warp slots.

kernels. This architectural coupling limits runtime policy flexibility, programmability, and adaptability in GPU-centric systems.

2.3 Limitations of Existing Extensibility Mechanisms

Existing approaches face limitations:

Driver-Level Kernel-Space Policies. Recent research has introduced driver-level resource control by moving policy decisions into kernel drivers or intercepting driver-level events. Systems like TimeGraph [26], Gdev [27], GPRE-EMPT [14], LithOS [11], GCAPS [57], and kernel interception methods [62] embed customized scheduling and memory-management logic into kernel-space or user-space driver modules. Despite improved performance, these approaches embed static policies requiring kernel modifications, restricting dynamic adaptation and safe deployment of new policies.

Host User-Space Runtimes and Libraries Recent frameworks such as Paella [38], Pie [17], XSched [48], and KTransformers [7] expose programmability at the user-space layer. Paella provides software-defined GPU scheduling for model serving; Pie enables developers to customize inference loops and KV-cache management via “inferlets”; XSched supports flexible, user-defined scheduling through its XQueue

abstraction; and KTransformers exposes programmable hybrid CPU–GPU scheduling and expert-placement strategies. LithOS [11] also re-implements runtime stacks with OS-like abstractions in user-space.

Despite this programmability, user-space systems share three limitations. First, programmability remains application-bound: developers must write policies for a specific framework or runtime, often requiring code changes or porting applications onto specialized stacks, preventing reuse across ecosystems. Second, user-space runtimes lack global visibility and coordination across multi-tenant, multi-framework environments. Confined to isolated process silos, they cannot safely share memory or bandwidth, nor mitigate interference from co-located tenants. As a result, applications conservatively pre-allocate resources (e.g., GPU memory) at startup to avoid contention [58, 61]. Likewise, hardware partitioning such as NVIDIA’s Multi-Instance GPU (MIG) imposes fixed resource boundaries that user-space runtimes cannot dynamically rebalance. Third, user-space frameworks lack privileged access to low-level driver mechanisms, leaving policies constrained to coarse-grained GPU kernel launch boundaries. This precludes precise preemption for latency-sensitive tasks and prevents fine-grained compute–transfer overlap. Moreover, they cannot execute policy logic on latency-critical paths (e.g., page-fault handlers), nor exploit GPU-side warp-level or barrier-level access patterns.

Binary Instrumentation and Profiling Tools on Device Tools such as NVBit [54] and Neutrino [22] inject logic directly into GPU binaries. NVBit supports dynamic SASS-level instrumentation, while Neutrino provides instruction-level profiling via programmable assembly-layer probes with an eBPF-like interface. Despite high kernel visibility, these tools face three limitations. First, they lack safety guarantees, as injected logic can cause crashes or deadlocks. Second, they target offline profiling rather than online policy enforcement, operating in read-only modes that cannot control memory placement or thread scheduling. Third, lacking host-side integration, they cannot coordinate policies across processes. Additionally, assembly-level instrumentation restricts expressiveness and portability [22]. Recent approaches like eGPU [59] extend eBPF to GPU instrumentation but remain limited to

observability with prohibitive overhead (see §6).

Host-Based Kernel Programmability (CPU eBPF) CPU-side eBPF has been used to customize OS scheduling and resource management (e.g., `sched_ext`, `cache_ext`). However, extending this model to GPUs faces two barriers. First, host-based eBPF treats the GPU as a black box: it can trace driver commands but cannot observe internal execution state (e.g., warp divergence), preventing reaction to on-chip bottlenecks. Second, unlike the Linux CPU scheduler refactored to support `sched_ext`, monolithic GPU drivers expose no safe programmable extension points. Their resource management mechanisms are tightly coupled to low-level hardware interactions (e.g., MMU invalidation), and naïvely exposing such controls risks resource leaks, hardware hangs, or kernel panics.

3 Design Principles

gBPF follows three design principles:

Principle 1: OS-level, Transparent and Dynamically Programmable Interface. GPU resource management policies should reside at an OS-level interface, rather than within vendor drivers or application-specific runtimes. Policies must be dynamically separable from mechanisms, runtime-updateable without application restarts, and enforceable across unmodified GPU workloads.

Principle 2: Cross-layer host–device policy abstraction. Host and GPU devices should not be managed as separate execution domains. Instead, the system should provide a unified programming abstraction and shared control plane across CPU and GPU, allowing consistent policy decisions (e.g., memory placement, scheduling).

Principle 3: SIMT-aware static safety and efficiency. Device-side policies must align with GPU SIMT execution semantics. The system should perform static, load-time verification and optimization, ensuring warp-uniform control flow, bounded resource usage, memory safety, and bounded overhead.

4 gBPF Design

4.1 Challenges

We elaborate on the three challenges introduced in §1.

4.1.1 C1: Lack of a Safe and Expressive OS-Level GPU Policy Interface

GPU resource management needs to run at the driver layer to be effective, but GPU drivers were not designed to expose a programmable interface. Exposing low-level mechanisms (page tables, command buffers, interrupt handlers) to eBPF would give policies expressiveness but compromise driver stability and tenant isolation. Conversely, constraining programmability to high-level abstractions ensures safety but limits the expressiveness needed for complex memory placement and scheduling decisions. Thus, the challenge lies in carving out a narrow, stable, and verifiable GPU policy interface: expressive enough to implement policies, yet constrained enough to prevent buggy policies from corrupting device state or crashing the kernel.

4.1.2 C2: Mismatch Between CPU Extension Semantics and the GPU SIMT Execution Model

Applying existing CPU-oriented eBPF mechanisms to GPU device is challenging due to semantic mismatches with the GPU SIMT execution model. CPU-side eBPF assumes single-threaded execution, whereas GPU kernels execute in warps of 32 threads that must follow warp-uniform control paths. Running scalar eBPF logic per thread introduces warp divergence, serialization overhead, and deadlock risks from thousands of concurrent memory operations unchecked by the CPU-side verifier. Furthermore, GPUs lack isolation and recovery points, meaning an unbounded loop or invalid access in policy logic can halt the entire device. This semantic mismatch makes reuse of CPU-oriented eBPF inefficient and unsafe.

4.1.3 C3: Absence of Efficient Mechanisms for Host–Device Shared State

GPU policies require coordinated host–device resource management, which today lacks efficient and consistent abstractions for shared policy state. GPU-accessible host memory exhibits orders-of-magnitude higher latency than GPU global memory, while GPU memory is capacity-limited and optimized for throughput. Meanwhile, CPU and GPU components operate asynchronously at different timescales, causing policy decisions to rely on stale or inconsistent state unless synchronized.

4.2 High-Level Architecture

Figure 3 shows the architecture of gBPF.

User-space Control Plane. Policy authors use standard eBPF tooling (e.g., `clang/libbpf`, `bpftool`) to build policy bytecode. The control-plane links against a loader library that installs policies and configures eBPF maps. It enables

4.3.2 Scheduling Interface

gBPF exposes scheduling policy control at two levels: hardware-queue lifecycle (host-side) and thread-block scheduling (device-side). The host participates in GPU queue admission and priority control through policies. Policies set queue priority, timeslice, and may reject or defer queue creation. Properties are written directly into firmware-visible structures, ensuring enforcement beyond user-space hints. Policies may also invoke the `gdrv_sched_preempt` kfunc to trigger cooperative preemption through the driver's native context-switch mechanism.

Within the GPU, gBPF provides a work-stealing thread-block scheduler. GPU kernels expose their logical work units; persistent worker blocks select and process units while device-side eBPF handlers steer scheduling decisions via `gdev_block_ctx`. The verifier ensures that scheduling decisions respect GPU kernel invariants (e.g., selecting only local work units). gBPF can inject this scheduler via binary rewriting or allow developers to integrate it.

```
1 // Host-side policy handlers for GPU hardware
2 // queue scheduling
3 struct gdrv_sched_ops {
4     // Hardware queue creation; set prio/timeslice,
5     // return 0 to accept or <0 to reject/defer
6     int (*queue_create)(gsched_queue_ctx_t *ctx);
7     // Hardware queue destruction; cleanup policy
8     void (*queue_destroy)(gsched_queue_ctx_t *ctx);
9 };
10
11 // Host-side kfunc
12 bool gdrv_sched_preempt(gdrv_preempt_ctx_t *ctx);
13
14 // Device-side block scheduling handlers
15 struct gdev_sched_ops {
16     // Worker block starts/exits new work unit
17     void (*enter)(gdev_block_ctx_t *ctx);
18     void (*exit)(gdev_block_ctx_t *ctx);
19
20     // Device function starts/ends
21     void (*probe)(gdev_block_ctx_t *ctx);
22     void (*retprobe)(gdev_block_ctx_t *ctx);
23
24     // Return whether to steal work (TB scheduler)
25     bool (*should_try_steal)(gdev_block_ctx_t *ctx);
26 };
```

4.4 Runtime Verification and Optimizations

gBPF policies execute directly within privileged GPU driver and kernel contexts, making safety guarantees critical. We adopt a standard eBPF threat model: we trust system administrators loading policies, but consider policy code potentially buggy or misconfigured. The trusted computing base (TCB) includes the kernel and gBPF kernel module, GPU compiler backend, and GPU driver/firmware. Policy authors only interact with eBPF and do not write native device code. For host-side verification, we reuse the existing Linux kernel eBPF verifier unchanged, relying on standard type-checking, bounded-loop, and memory-safety constraints. gBPF-specific

kfuncs and hooks are declared via BPF Type Format (BTF) metadata.

4.4.1 SIMT-aware Verification

To solve the SIMT mismatch described in C2, gBPF introduces a SIMT-aware static verification model enforcing warp-level uniformity constraints to ensure efficient and safe GPU-side policy execution. The key insight is that we distinguish *warp-uniform* values (identical across warp threads, e.g., region identifiers) from *lane-varying* values (thread-local GPU state). Policies may compute freely using lane-varying values, but these must not influence control-flow decisions or directly produce side effects visible outside a warp. To guarantee warp-level coherence, gBPF mandates that branch conditions, loop bounds, and map-update keys be warp-uniform or derived via warp-level aggregation. Additionally, gBPF prohibits GPU-wide barriers, global synchronization primitives, and non-uniform atomic operations, and imposes resource budgets per policy hook to bound memory and thread resource usage. These constraints ensure safe policy integration that respects GPU hardware execution semantics.

4.4.2 Warp-level Execution Optimizations

Direct scalar execution of eBPF after SIMT verification on GPU threads still causes resource overhead (e.g. duplicate memory update to maps) due to the SIMT. To resolve this, gBPF introduces a warp-level aggregated execution model, executing policy logic once per warp using a designated "warp leader" (typically lane 0). Each hook invocation conceptually follows a two-phase approach: each lane independently computes lane-local contributions (e.g., effective addresses, counters) without influencing control flow or directly updating shared state. These contributions are then aggregated at warp granularity. Subsequently, the warp leader executes the policy handler once using aggregated inputs and warp-uniform context. The resulting decision is broadcast back to all lanes. This design minimizes GPU overhead while preserving eBPF's scalar semantics. To further reduce runtime overhead, gBPF employs compiler-level optimizations (specialization and inlining) tailored for warp-level execution.

4.4.3 Hierarchical Cross-layer Maps for State Coordination

GPU policy decisions require coherent sharing of policy state across asynchronous CPU and GPU contexts. To transparently bridge CPU-GPU memory hierarchies, gBPF introduces cross-layer maps. Logically, each map is a single unified key-value store accessible from host-side driver hooks, GPU-side device policies, and user-space control planes, and verifiable through eBPF's existing model. Physically, maps are realized as hierarchical shards across host DRAM, GPU global

memory, and GPU-local storage (e.g., shared memory, SM-local). The runtime dynamically determines shard placement based on access frequency, latency, and capacity constraints. Following eBPF’s soft-state philosophy, gBPF maps provide relaxed, eventual consistency. Instead of strong global ordering, our runtime periodically merges GPU-local map shards into canonical snapshots at synchronization points, such as GPU kernel completion boundaries. This snapshot-based model is sufficient for policy decisions relying on approximate statistics. Occasional staleness affects decision optimality but cannot violate correctness invariants such as memory integrity, which remain enforced by the GPU driver and hardware MMU.

5 Implementation

5.1 Components Overview

We implemented gBPF on Linux 6.15, extending the kernel with a standalone file (~1 KLOC) that handles hook registration and verifier extensions. We integrated lightweight instrumentation (~100 LOC) into NVIDIA’s open GPU kernel modules to expose memory management and scheduling decision points (§5.2). Our userspace loader and JIT infrastructure (~10 KLOC) builds upon bpftime’s framework, while an LLVM-based backend (~1 KLOC) translates the restricted gBPF eBPF instruction subset into GPU device code (PTX) (§5.3).

5.2 Host-side Integration

gBPF extends the NVIDIA Open GPU Kernel Modules [39], instrumenting the UVM module at page fault handler and prefetch logic. At each hook, the driver constructs a context containing fields (e.g., region range aligned to 2MB GPU physical chunks, VA blocks or 4KB pages, fault address, block size) and invokes an attached eBPF handler. Each handler returns a decision to execute or bypass default kernel logic. The kernel maintains a doubly-linked eviction list; handlers may reorder regions via provided helpers, but the kernel retains eviction authority under memory pressure.

We instrument Time-Slice Group (TSG) lifecycle events, including initialization (`queue_create`) and teardown (`queue_destroy`). The eBPF policy can configure scheduling parameters such as TSG priority, timeslice duration, and runlist interleave frequency, and can cooperatively preempt active TSGs via the `gdrv_sched_preempt` kfunc. Our kernel module extends the Linux eBPF verifier by registering GPU-specific `struct_ops` and `kfunc` interfaces, leveraging BTF annotations to enforce type and memory safety.

5.3 GPU Runtime

After the Linux kernel verifies a gBPF device eBPF program, our loader forwards bytecode and BTF metadata to a userspace backend, translating the restricted eBPF subset into GPU PTX code. We reuse Linux’s eBPF verifier to enforce standard memory safety, bounded loops, and type correctness. An additional verifier pass leverages BTF metadata annotations that mark warp-uniform fields: at load time, our verifier performs dataflow analysis on eBPF bytecode, propagating warp-uniformity through arithmetic, logical, and memory operations. Control-flow constraints are enforced by traversing the policy’s CFG to reject lane-varying dependencies in branch predicates or loop bounds. Map-update instructions must use warp-uniform keys derived from uniform registers or warp-level reductions. GPU-wide barriers, global synchronization primitives, and non-uniform atomics are disallowed by pattern-matching. Each hook type carries resource budgets limiting instructions, helper invocations, and memory operations.

Before each GPU kernel runs, gBPF dynamically intercepts CUDA runtime APIs to extract GPU kernel PTX, rewrite it with binary trampolines, and load instrumented GPU kernels back without recompilation or restarting the application. These trampolines, placed at GPU kernel entry, selected memory instructions (such as global loads and atomic operations), and execution-phase boundaries, insert prologue/epilogue logic for SIMT register preservation. In each trampoline, per-lane inputs are aggregated using `__ballot_sync` and `__shfl_sync`, with a warp leader selected via `__ffs(__activemask())` to execute the scalar eBPF handler once per warp, broadcasting results via shuffle instructions; verified eBPF code executes after kernel launch, supported by pre-compiled GPU helper functions. During JIT compilation, we inline helper functions and map accesses to reduce call overhead. For scheduling, gBPF implements policies using persistent GPU workers as long-lived kernels that poll and claim logical work units; on Blackwell GPUs, we use cluster launch control APIs to schedule thread blocks. For memory management instrumentation, gBPF attaches eBPF policy at memory instructions, GPU kernel function boundaries, and tracepoints to issue policy hints to the host driver. Unlike eGPU [59] which relies on atomic synchronization, gBPF avoids cross-SM synchronization primitives to prevent hardware stalls.

Hierarchical Maps and Cross-Domain State We use eBPF maps in GPU-accessible memory as a shared state abstraction between host-driver and GPU-device contexts. Map shards are placed according to access patterns: cold global state resides in host DRAM and is periodically refreshed from GPU-side snapshots; frequently accessed state resides in GPU global memory and is asynchronously flushed to host; hot per-warp or per-SM data is cached in GPU shared memory for low

latency. Consistency across shards is maintained via snapshot-based aggregation at GPU kernel completion boundaries. A runtime daemon asynchronously flushes GPU-local shards to host-visible canonical map instances, providing coherent snapshots to host-side policies without synchronization overhead. Shard updates occur strictly at warp granularity through warp-leader threads without GPU atomic operations. For example, GPU-side handlers accumulate per-region access counters in warp-local registers, aggregate them per warp, and store results into GPU-local shards. The periodic snapshot flush transfers these shards to the host for policy decisions.

6 Evaluation

We evaluate gBPF using three research questions:

- **RQ1 (Single-Tenant Management):** How much performance gain can gBPF’s programmable memory and scheduling policies provide on oversubscribed single-tenant workloads?
- **RQ2 (Multi-Tenant Management):** Does gBPF’s cross-layer policy runtime improve tail latency, throughput, and resource fairness compared to user-space and global policies in multi-tenant settings?
- **RQ3 (Programmability & Mechanism Overhead):** Is gBPF sufficiently programmable to support diverse policies, and what is the overhead of its core mechanisms and observability capabilities?

These questions cover single-tenant optimization (RQ1), multi-tenant coordination (RQ2), and system costs (RQ3).

6.1 Methodology and Setup

We evaluate gBPF on two machines: Server A with Intel Core Ultra 9 285K (24 cores), 128 GB DDR5 RAM, and NVIDIA RTX 5090 GPU; Server B with dual Intel Gold 6138 (80 cores), 256 GB RAM, and NVIDIA P40 GPU. Unless otherwise specified, experiments run on Server A, averaging 10 trials using geometric mean. We compare against default UVM, UVM with user-space hints, native GPU scheduler, and framework-managed offloading where applicable. We use PyTorch (version 2.9.0) and vLLM (version 0.11.0) with custom allocator, llama.cpp (version 7101), and Faiss (version 1.13.0) with UVM build config for our evaluation. All tests with gBPF policies require no application modifications.

6.2 RQ1: Single-Tenant Memory and Scheduling Policies

We evaluate gBPF’s memory policy interface on oversubscribed single-tenant setups where memory placement and prefetching dominate performance.

6.2.1 Microbenchmarks

Memory Policy Microbenchmarks. We evaluate host-device prefetch coordination using a modified vector-add GPU kernel [41] with stride access pattern and 40GB working set ($1.25\times$ oversubscription). Device-side L2 prefetch instructions (`prefetch.global.L2`) trigger non-blocking page faults, while host-side callbacks extend prefetching to additional pages. With device-only prefetch and default host prefetch at thread entry, we achieve $1.34\times$ speedup; combined host-device stride-based eBPF prefetch achieves $1.77\times$. However, sequential prefetch degrades performance by 8% due to pattern mismatch, demonstrating that gBPF’s cross-layer design enables workload-aware prefetching where policy selection matters.

Single-Tenant Block Scheduler. We apply a gBPF-based block scheduler driven by the GPU-side interface (Section 4.3.2) to workloads with load imbalance. Using cluster-style launch where persistent worker blocks pull work units via `should_try_steal`, we evaluate three policies: Fixed-Work (static block assignment, no scheduler), Greedy (always-steal), and LatencyBudget (workload-aware with per-block time limits). Figure 4 compares these policies across two GEMM workloads. Under moderate imbalance (Figure 4a), both Greedy and LatencyBudget reduce latency by $\sim 11\%$, as always-steal scheduling reclaims wasted tail time. Under heavy-tail workloads where 10% of blocks perform 100–200 $\times$ more work (Figure 4b), Greedy increases latency by 20% due to contention on heavy blocks. LatencyBudget, which caps per-block stealing time, matches fixed-work baseline. This highlights gBPF’s programmability: different workloads require different scheduling strategies.

6.2.2 Case Studies

Expert Offloading (llama.cpp GPT-OSS-120B). We evaluate gBPF on GPT-OSS-120B MXFP4 MoE (59 GiB, 116.83B parameters) in llama.cpp on RTX 5090 (32GB). The model requires $1.84\times$ oversubscription, necessitating CPU-GPU memory tiering. We compare five configurations: framework-managed CPU offloading (`ncmoe=32/64`), default UVM, modify application to add user-space hints (`cudaMemAdvise`) with UVM, and gBPF with eBPF-based expert prefetching. Figure 5 shows prefill and decode throughput. For decode (memory-bound), gBPF achieves **$4.8\times$ speedup** over framework offloading. For prefill (compute-bound), framework offloading achieves 13% higher throughput, but decode dominates end-to-end latency, so the $4.8\times$ decode improvement outweighs the 13% prefill overhead. Static hints can affect default LRU algorithms to pin most of the memory on GPU to improve performance, but still falls behind our policy, confirming that UVM requires new algorithms. The key insight is that MoE workloads exhibit pre-

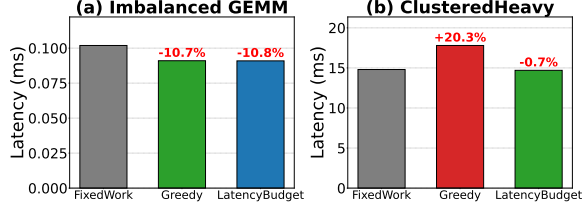


Figure 4: gBPF block-scheduling policies across workload regimes. No single policy dominates: always-steal works well under moderate imbalance (a) but becomes pathological under clustered heavy tails (b), where LatencyBudget matches baseline performance.

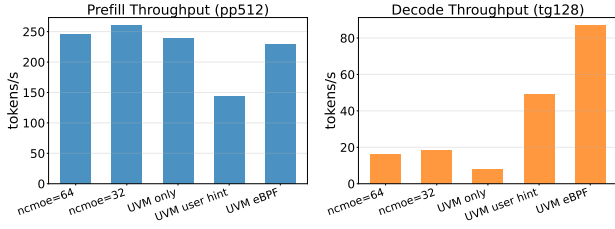


Figure 5: Prefill (pp512) and decode (tg128) throughput for GPT-OSS-120B MoE (59 GiB) on RTX 5090 (32GB). gBPF eBPF prefetching achieves $4.8\times$ speedup on decode (memory-bound) over framework offloading while maintaining competitive prefill performance.

dictable stride patterns during weight access and non-uniform page-level access frequency. gBPF uses stride prefetch to overlap data transfer with computation, and LFU eviction to retain frequently-accessed pages. Unlike framework-level offloading that often migrates experts as atomic units, gBPF operates at page granularity, enabling finer-grained compute-transfer overlap.

KV-cache Offloading (vLLM Qwen-30B MoE). We evaluate on the Qwen-30B FP8 MoE model (≈ 30 GB) on RTX 5090 (32GB). The model alone nearly fills GPU VRAM, so default no-offload configuration OOMs under load. We stress KV-cache growth by sending 100 concurrent requests from ShareGPT [55] (single-round, no prefix caching), resulting in total memory footprint of 36–40GB (configured KV-cache at ≈ 60 K tokens). Existing solutions typically offload either KV-cache or MoE experts, but not both; in contrast, UVM attempts on-demand migration of both, leading to thrashing.

We compare vLLM’s CPU-offload mode (`-cpu-offload-gb 8`) against gBPF with UVM and a sequential prefetch policy. Under memory pressure, the key challenge is managing two competing data types: KV-cache and model weights. KV-cache exhibits temporal locality (recent tokens accessed more frequently during attention) with per-request spatial locality, while expert weights show stride patterns during matrix operations. gBPF

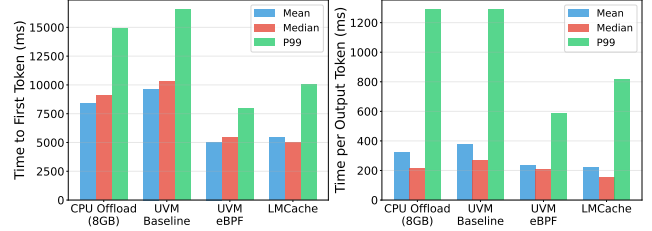


Figure 6: Time-to-first-token and decoding throughput on Qwen-30B FP8 MoE with 100 concurrent requests (≈ 60 K tokens KV-cache). gBPF KV-aware sequential prefetch improves mean and p99 TTFT by $1.7\text{--}2\times$ and decoding throughput by $1.3\times$ over vLLM CPU-offload.

applies prefetching with adaptive aggressiveness based on PCIe utilization and memory region (stride-based for weights, sequential for KV-cache), and LFU eviction to retain frequently-accessed pages from both types. This prevents mutual thrashing that occurs with UVM’s default LRU when KV-cache growth displaces hot weight pages or vice versa.

gBPF improves mean and p99 time-to-first-token by $1.7\text{--}2\times$ and decoding throughput by $1.3\times$ over vLLM’s default framework-managed offload (Figure 6), matching the state-of-the-art KV cache offload framework LMCACHE [9] with better tail latency. Notably, UVM without gBPF policies performs worse than vLLM’s offload, while gBPF is $2\text{--}3\times$ faster, showing driver-level policies can outperform framework offloading with better tail latency.

Graph Neural Networks (GNN Training). We evaluate gBPF on PyTorch-based GCN training with random graphs of varying sizes (1M–15M nodes, 10 edges/node). Figure 7 shows epoch time across five configurations: native GPU allocation (no UVM), UVM with and without user-space prefetching (`cudaMemPrefetchAsync`), and each UVM configuration with gBPF eBPF-based optimization.

With UVM enabled via a custom allocator, each epoch allocates memory on CPU and migrates to GPU on demand, incurring overhead. gBPF uses sequential prefetch with uprobe to trace PyTorch memory allocations and prefetch in advance, using history patterns to determine prefetch range. Users can also express prefetching via `cudaMemPrefetchAsync` to eliminate page faults by migrating pages before GPU access, achieving $5.5\times$ speedup at moderate oversubscription, but this requires application modification. When user-expressed prefetching is disabled, eBPF optimizes page fault handling by aggressive prefetching, achieving $2.65\times$ speedup without modifying applications. Even with user-expressed prefetching, runtime page faults and eviction occur under memory pressure, and gBPF’s combined approach achieves $1.44\times$ additional speedup by prefetching larger regions and reducing blocking faults. Native GPU allocation (no UVM) achieves the highest performance when data fits in memory (1.89 s at

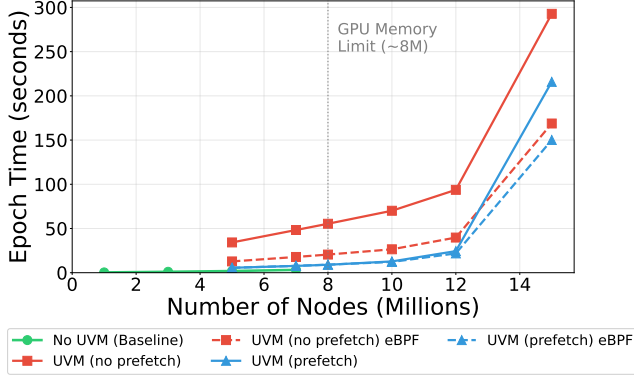


Figure 7: GCN training epoch time vs graph size. Native GPU allocation (green) OOMs beyond memory capacity. User-space prefetching (blue) eliminates page faults but requires application modification; gBPF eBPF prefetching (red) achieves similar benefits transparently. Combined approaches (dashed blue) provide additional benefits under severe oversubscription.

5M nodes) but fails with OOM beyond ~ 8 M nodes. UVM with gBPF prefetching enables training at 15M nodes ($2.17\times$ oversubscription) with acceptable overhead.

Faiss Vector Search. We evaluate gBPF on vector similarity search using the SIFT dataset with an IVF4096, Flat index, scaling from 20M (9.5GB, fits in memory) to 50M (24GB, near boundary) and 100M (48GB, exceeds 32GB GPU memory). IVF indexes have a two-level structure: frequently-accessed centroids and larger posting lists. For oversubscribed datasets, gBPF prefetches posting lists sequentially rather than UVM’s default address-tree-based policy; device-side can also trigger prefetching of corresponding posting lists. For index construction, K-means iterations produce sequential scans that gBPF detects and prefetches using stride prediction. gBPF reduces build time by 21–29%, with benefits scaling from 27% to 40% as memory pressure grows. For search workloads, gBPF reduces latency by 10–16% across different `nprobe` settings despite the random access patterns inherent in ANN search.

Across these applications, programmable region-based placement and prefetch policies in gBPF exploit workload structure (experts, KV-cache, graph structure, index layout) that existing UVM policies and framework-level offload mechanisms cannot leverage.

6.3 RQ2: Multi-Tenant Memory, Bandwidth, and Scheduling

We evaluate gBPF on multi-tenant GPU workloads, comparing per-tenant policies against global and framework-managed approaches.

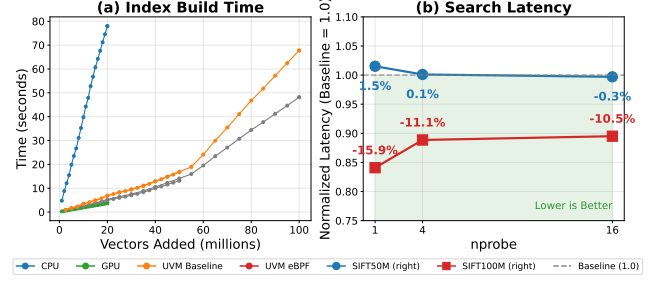


Figure 8: Faiss index build time and query throughput under oversubscription. gBPF adaptive prefetch reduces build time by 21–29% and query latency by 10–16% compared to default UVM.

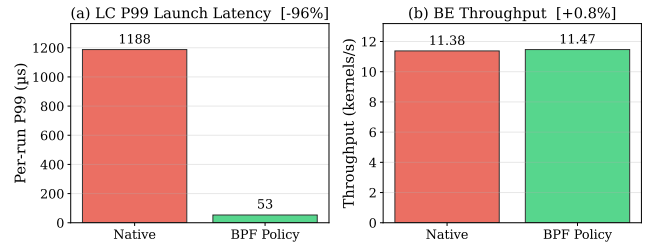


Figure 9: Multi-tenant GPU scheduling with gBPF BPF struct_ops policies on compute-bound workloads. gBPF reduces latency-critical (LC) P99 launch latency by 95% while maintaining best-effort (BE) throughput.

6.3.1 Microbenchmarks

Compute-bound Timeslice Scheduler (LC + BE). We evaluate gBPF’s BPF struct_ops scheduling policies on *compute-bound* multi-tenant workloads where the bottleneck is GPU SM time rather than memory bandwidth. 2 latency-critical (LC) processes and 4 best-effort (BE) processes run concurrently, each with 4 CUDA streams submitting 50 compute kernels. We compare the native scheduler against gBPF with differentiated timeslices (LC: 1s, BE: 200 μ s) using a gpreempt-style preemption policy. Figure 9 shows that gBPF reduces LC P99 launch latency by 95%, demonstrating scheduling stability. BE throughput remains unchanged, confirming that gBPF reduces LC tail latency without sacrificing BE throughput.

Memory-bound Memory Priority Differentiation. We evaluate gBPF’s memory policies for priority differentiation on *memory-bound* workloads under UVM oversubscription using HotSpot [6] (spatial locality), GEMM from PolyBench [18] (compute-intensive), and K-Means [19] (sparse access). Each experiment runs two concurrent processes competing for GPU memory. Policy labels use priority values 0–100 (lower value = higher priority): `Prefetch(lo, hi)` prefetches

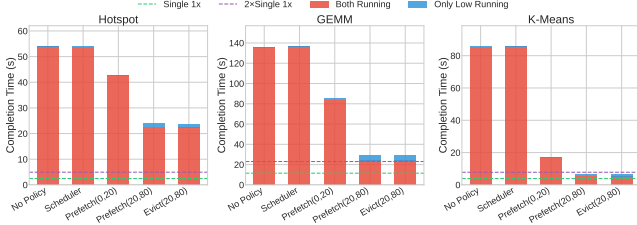


Figure 10: Multi-tenant memory priority differentiation on oversubscribed UVM workloads. gBPF memory policies improve total completion time by 55–92% and enforce priority differentiation; scheduler policies are ineffective on these memory-bound workloads. Single 1×: ideal single-process time; 2×Single 1×: theoretical lower bound.

pages for processes in that range, while `Evict(lo,hi)` selects eviction victims accordingly. We compare against: (1) no-policy execution, (2) single-process runtime (Single 1×), and (3) theoretical optimum (2×Single 1×).

Figure 10 shows that without policies, both processes degrade severely due to memory thrashing. With gBPF memory priority policies, total completion time improves by 55–92%, and high-priority processes complete 6–19% faster. Critically, for these *memory-bound* workloads, GPREEMPT-style [14] scheduler timeslice policies are *ineffective* (<1% improvement) because the bottleneck is UVM page faults and PCIe bandwidth, not GPU compute time. This contrasts with compute-bound workloads where scheduler policies are effective. gBPF’s layered runtime allows applying appropriate policies based on workload characteristics.

6.3.2 Case Studies

We apply the policy primitives from the microbenchmarks to realistic multi-tenant scenarios.

Two-Tenant: High-Priority LLM Inference + Background Training. We run two tenants sharing a single GPU: T1 is a high-priority `llama.cpp` inference service (LC workload) serving 100 ShareGPT prompts at 0.2 RPS, and T2 is a background PyTorch GNN training job (BE workload) with 8M nodes requiring 36GB peak memory (oversubscribed). We compare default UVM (driver FIFO/LRU) against gBPF per-tenant memory policies where the LC tenant receives prefetch priority and the BE tenant yields bandwidth during inference.

Figure 11 shows the results. gBPF achieves mutual improvement for both tenants: LC latency reduces TPOT by 40–45% and TTFT by 14–20%, while GNN training simultaneously improves by 28%, approaching the ideal fair share baseline. This shows gBPF reduces contention for both workloads rather than favoring one over the other.

gBPF’s per-tenant policies improve both tail latency and

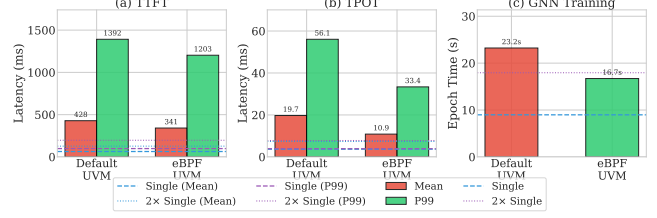


Figure 11: Two-tenant co-location: `llama.cpp` inference (LC) + GNN training (BE). gBPF achieves mutual improvement: LC latency better than single-tenant performance while BE training improves by 28%. Tenant-aware policies reduce contention rather than prioritizing one workload at the expense of another.

Policy	Tag	Domain	LOC
Global FIFO Eviction	Memory Eviction	Host	145
Global LFU Eviction	Memory Eviction	Host	304
Multi-tenant LRU	Memory Multi-tenant	Host	472
Adaptive Prefetch	Seq. Memory Prefetch	Host	375
Stride Prefetch	Memory Prefetch	Host	472
GPU L2 Stride Prefetch	Memory Prefetch	Device	45
Tree-based Prefetch	Memory Multi-tenant	Host	454
Dynamic Timeslice	Scheduling	Host	408
Preemption Control	Scheduling	Host	925
MaxSteals (CLC)	Scheduling Block	Device	16
LatencyBudget (CLC)	Scheduling Block	Device	19

Table 1: Policy support matrix. Domain indicates where the policy executes: Host (driver eBPF), Device (GPU eBPF), or both.

fairness, benefiting high-priority tenants while reducing memory thrashing for background jobs.

6.4 RQ3: Programmability and Mechanism Overhead

We evaluate gBPF’s programmability by examining policy complexity and reuse, and quantify the efficiency of its core mechanisms and observability capabilities.

Policy Building Blocks. Table 1 shows policy building blocks with lines of code and execution domain. These components can be composed to construct policies; for example, multi-tenant benchmark in RQ2 combines Quota LRU, Priority Tree Prefetch, and Dynamic Timeslice. All policies attach to unmodified frameworks via runtime instrumentation, requiring only tens to hundreds of lines.

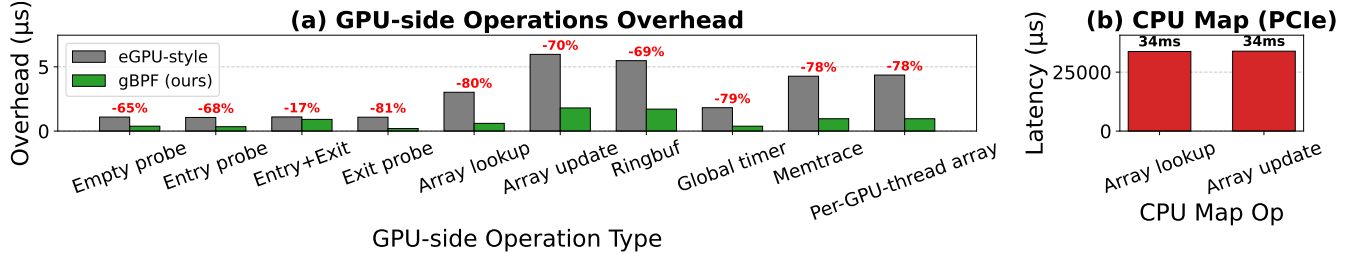


Figure 12: Device-side microbenchmark overhead. (a) GPU-side operation overhead comparing eGPU-style naive injection vs gBPF’s SIMT-aware execution. (b) eGPU CPU map access latency via PCIe, motivating gBPF’s hierarchical map design.

Tool	LOC	Domain	gBPF	NVBit
kernelretnoop	153	Device	8%	85%
threadhist	89	Device	3%	87%
launchlate	347	Host+Device	14%	93%

Table 2: gBPF device-side observability tools and overhead comparison with NVBit.

6.4.1 Host Runtime Overhead

We quantify the overhead of the gBPF driver runtime independent of policy logic using GEMM from PolyBench [18] and HotSpot [6] ($1.1\times$ oversubscription, 35 GB working set) on RTX 5090. With hooks enabled but no policy attached, the overhead is less than 0.2%.

6.4.2 Low-overhead GPU Observability

We evaluate device-side observability tools built on gBPF (Table 2) on NVIDIA P40 GPU using llama.cpp prefill (Llama 1B, IO3, sequence-mixed). Tools include `kernelretnoop` for per-thread block finish timestamp recording, `threadhist` for load imbalance detection, and `launchlate` for GPU kernel launch latency measurement. Overhead is measured as token/s degradation during prefill. These tools show significantly lower overhead than NVBit [54] (3–14% vs 85–87%) due to gBPF’s warp-uniform execution avoiding divergent control flow and unnecessary memory accesses.

Device-side Runtime Overhead and Optimization. We evaluate device-side gBPF mechanisms using a vector-add microbenchmark (32 elements, 10K iterations) from `cuda-sample` [41] on RTX 5090. Figure 12(a) compares gBPF’s SIMT-aware warp-level execution against an eGPU baseline [59] that injects eBPF bytecode without SIMT-specific optimizations. The eGPU-style approach incurs overhead due to per-thread execution and uncoalesced memory access, while gBPF’s warp-uniform execution and coalesced map access reduce overhead by 60–80% across operations. Figure 12(b) shows CPU map access via PCIe is $6000\times$ slower than GPU-side operations, motivating gBPF’s hierarchical map design

that keeps hot state on GPU. We do not compare policy-level performance against eGPU or Neutrino because they are limited to read-only observability (§2.3).

7 Portability

gBPF is designed for portability, though our current implementation targets NVIDIA GPUs. NVIDIA’s production drivers bypass Linux’s standard kernel interfaces, but our design aligns with generic Linux abstractions. On the host side, for memory management we follow Linux’s Heterogeneous Memory Management (HMM) and its `migrate_vma` interface, already adopted by AMD ROCm with XNACK retryable faults. For scheduling, we abstract GPU queues based on Linux DRM scheduler’s `drm_sched_entity`, mapping to NVIDIA TSGs and potentially AMD user-mode queues [3, 31–33]. On the device side, our compiler generates vendor-neutral eBPF bytecode using SPIR-V, enabling future compatibility with AMD ROCm and Intel Level Zero. End-to-end portability requires vendor-specific runtime support and hardware tuning, which remain future work [10, 42].

8 Conclusion

We presented gBPF, a cross-layer, verifier-backed policy runtime that treats the GPU driver and device as a single programmable OS subsystem. gBPF exposes stable attach points for memory placement and scheduling, executes the same restricted eBPF IR on host and device, and shares state through cross-layer BPF maps, enabling low-overhead observability and adaptive policies. Across PyTorch, llama.cpp, vLLM, and Faiss, gBPF-based policies transparently improve throughput by up to $4.8\times$ and reduce tail latency by up to $2\times$ relative to static heuristics, while instrumentation-only deployments incur low overhead. gBPF demonstrates that kernel extensibility is an effective approach for GPU management.

References

- [1] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems*, 6:351–366, 2024.
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024.
- [3] AMD ROCm Documentation. *GPU memory*, 2024. <https://rocm.docs.amd.com/en/docs-6.2.0/conceptual/gpu-memory.html>.
- [4] Maximilian Bachl, Joachim Fabini, and Tanja Zseby. A flow-based ids using machine learning in ebpf. *arXiv preprint arXiv:2102.09980*, 2021.
- [5] Jiashen Cao, Rathijit Sen, Matteo Interlandi, Joy Arulraj, and Hyesoon Kim. Gpu database systems characterization and optimization. *Proceedings of the VLDB Endowment*, 17(3):441–454, 2023.
- [6] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W Sheaffer, Sang-Ha Lee, and Kevin Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE international symposium on workload characterization (IISWC)*, pages 44–54. Ieee, 2009.
- [7] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, et al. Ktransformers: Unleashing the full potential of cpu/gpu hybrid inference for moe models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 1014–1029, 2025.
- [8] Xuhao Chen, Roshan Dathathri, Gurbinder Gill, and Keshav Pingali. Pangolin: An efficient and flexible graph mining system on cpu and gpu. *Proceedings of the VLDB Endowment*, 13(8):1190–1205, 2020.
- [9] Yihua Cheng, Yuhan Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. Lmcache: An efficient kv cache layer for enterprise-scale llm inference. *arXiv preprint arXiv:2510.09665*, 2025.
- [10] LLVM Community. Rfc: Spir-v ir as a vendor agnostic gpu representation. LLVM Discourse, 2025. <https://discourse.llvm.org/t/rfc-spirv-ir-as-a-vendor-agnostic-gpu-representation/85115>.
- [11] Patrick H Coppock, Brian Zhang, Eliot H Solomon, Vasilis Kypriotis, Leon Yang, Bikash Sharma, Dan Schatzberg, Todd C Mowry, and Dimitrios Skarlatos. Lithos: An operating system for efficient machine learning on gpus. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 1–17, 2025.
- [12] Christina Delimitrou and Christos Kozyrakis. Paragon: Qos-aware scheduling for heterogeneous datacenters. *Acm SIGPLAN Notices*, 48(4):77–88, 2013.
- [13] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and qos-aware cluster management. *ACM Sigplan Notices*, 49(4):127–144, 2014.
- [14] Ruwen Fan, Tingxu Ren, Minhui Xie, Shiwei Gao, Jiwei Shu, and Youyou Lu. {GPREENPT}:{GPU} preemptive scheduling made general and efficient. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 263–272, 2025.
- [15] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. {ServerlessLLM}:{Low-Latency} serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 135–153, 2024.
- [16] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. Megablocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems*, 5:288–304, 2023.
- [17] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. Pie: A programmable serving system for emerging llm applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 415–430, 2025.
- [18] Scott Grauer-Gray, Lifan Xu, Robert Searles, Sudhee Ayalasomayajula, and John Cavazos. Auto-tuning a high-level language targeted to gpu codes. In *2012 innovative parallel computing (InPar)*, pages 1–10. Ieee, 2012.
- [19] Yongbin Gu, Wenxuan Wu, Yunfan Li, and Lizhong Chen. Uvmbench: A comprehensive benchmark suite for researching unified virtual memory in gpus. *arXiv preprint arXiv:2007.09822*, 2020.
- [20] Jiaao He, Jiezhong Qiu, Aohan Zeng, Zhilin Yang, Jidong Zhai, and Jie Tang. Fastmoe: A fast mixture-of-expert training system. *arXiv preprint arXiv:2103.13262*, 2021.

- [21] Qinghao Hu, Meng Zhang, Peng Sun, Yonggang Wen, and Tianwei Zhang. Lucid: A non-intrusive, scalable and interpretable scheduler for deep learning training jobs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 457–472, 2023.
- [22] Songlin Huang and Chenshu Wu. Neutrino: Fine-grained {GPU} kernel profiling via programmable probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 331–355, 2025.
- [23] Suhas Jayaram Subramanya, Daiyaan Arfeen, Shouxu Lin, Aurick Qiao, Zhihao Jia, and Gregory R Ganger. Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 642–657, 2023.
- [24] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. Analysis of {Large-Scale}{Multi-Tenant}{GPU} clusters for {DNN} training workloads. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 947–960, 2019.
- [25] Jinghao Jia, YiFei Zhu, Dan Williams, Andrea Arcangeli, Claudio Canella, Hubertus Franke, Tobin Feldman-Fitzthum, Dimitrios Skarlatos, Daniel Gruss, and Tianyin Xu. Programmable system call security with ebpf. *arXiv preprint arXiv:2302.10366*, 2023.
- [26] Shinpei Kato, Karthik Lakshmanan, Ragunathan Rajkumar, Yutaka Ishikawa, et al. {TimeGraph}:{GPU} scheduling for {Real-Time}{Multi-Tasking} environments. In *2011 USENIX Annual Technical Conference (USENIX ATC 11)*, 2011.
- [27] Shinpei Kato, Michael McThrow, Carlos Maltzahn, and Scott Brandt. Gdev:{First-Class}{GPU} resource management in the operating system. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 401–412, 2012.
- [28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with page-dattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [29] Baolin Li, Tirthak Patel, Siddharth Samsi, Vijay Gadeppally, and Devesh Tiwari. Miso: exploiting multi-instance gpu capability on multi-tenant gpu clusters. In *Proceedings of the 13th Symposium on Cloud Computing*, pages 173–189, 2022.
- [30] Yinnian Lin, Lei Zou, and Xunbin Su. Towards sufficient gpu-accelerated dynamic graph management: Survey and experiment. *Proceedings of the VLDB Endowment*, 18(3):599–612, 2024.
- [31] The Linux Kernel Documentation. *GPU Shared Virtual Memory (GPU-SVM)*, 2023. Section “DRM GPU scheduler” and GPU-SVM overview, <https://docs.kernel.org/gpu/drm-mm.html>.
- [32] The Linux Kernel Documentation. *DRM GPU scheduler*, 2024. <https://docs.kernel.org/gpu/drm-mm.html>.
- [33] The Linux Kernel Documentation. *User Mode Queues*, 2024. <https://docs.kernel.org/gpu/amdgpu/userq.html>.
- [34] David Lo, Liqun Cheng, Rama Govindaraju, Parthasarathy Ranganathan, and Christos Kozyrakis. Heracles: Improving resource efficiency at scale. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, pages 450–462, 2015.
- [35] Meta and Linux Kernel Community. sched_ext: Extensible scheduler class. <https://github.com/sched-ext/scx>, 2023.
- [36] Kenneth Moreland, Christopher Sewell, William Usher, Li-ta Lo, Jeremy Meredith, David Pugmire, James Kress, Hendrik Schroots, Kwan-Liu Ma, Hank Childs, et al. Vtk-m: Accelerating the visualization toolkit for massively threaded architectures. *IEEE computer graphics and applications*, 36(3):48–58, 2016.
- [37] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. {Heterogeneity-Aware} cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498, 2020.
- [38] Kelvin KW Ng, Henri Maxime Demoulin, and Vincent Liu. Paella: Low-latency model serving with software-defined gpu scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 595–610, 2023.
- [39] NVIDIA. NVIDIA open gpu kernel modules, 2022. Accessed: 2025.
- [40] NVIDIA. Cuda profiling tools interface (cupti), 2025. Accessed: 2025-03-02.
- [41] NVIDIA. Nvidia cuda samples. <https://github.com/NVIDIA/cuda-samples>, 2025. Accessed: 2025-12-11.

- [42] oneAPI DPC++/LLVM. *Proposed design for offloading model*, 2023. <https://intel.github.io/llvm/design/OffloadDesign.html>.
- [43] James Jie Pan, Jianguo Wang, and Guoliang Li. Survey of vector database management systems. *The VLDB Journal*, 33(5):1591–1615, 2024.
- [44] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024.
- [45] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R Ganger, and Eric P Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, 2021.
- [46] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3505–3506, 2020.
- [47] John Ravi, Tri Nguyen, Huiyang Zhou, and Michela Becchi. Pilot: a runtime system to manage multi-tenant gpu unified memory footprint. In *2021 IEEE 28th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, pages 442–447. IEEE, 2021.
- [48] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. {XSched}: Preemptive scheduling for diverse {XPU}s. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 671–692, 2025.
- [49] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.
- [50] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [51] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. {USHER}: Holistic interference avoidance for resource optimized {ML} inference. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 947–964, 2024.
- [52] Foteini Strati, Xianzhe Ma, and Ana Klimovic. Orion: Interference-aware, fine-grained gpu sharing for ml applications. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 1075–1092, 2024.
- [53] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, pages 173–191, 2024.
- [54] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W Keckler. Nvbit: A dynamic binary instrumentation framework for nvidia gpus. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 372–383, 2019.
- [55] G. Wang. openchat: advancing open-source language models with conversational datasets. 2023. Describes the ShareGPT dataset of 70k user-shared conversations.
- [56] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D Owens. Gunrock: A high-performance graph processing library on the gpu. In *Proceedings of the 21st ACM SIGPLAN symposium on principles and practice of parallel programming*, pages 1–12, 2016.
- [57] Yidi Wang, Cong Liu, Daniel Wong, and Hyoseung Kim. Gcaps: Gpu context-aware preemptive priority-based scheduling for real-time tasks. *arXiv preprint arXiv:2406.05221*, 2024.
- [58] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, et al. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, 2018.
- [59] Yiwei Yang, Tong Yu, Yusheng Zheng, and Andrew Quinn. egpu: Extending ebpf programmability and observability to gpus. In *Proceedings of the 4th Workshop on Heterogeneous Composable and Disaggregated Systems*, pages 73–79, 2025.
- [60] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.

- [61] Peifeng Yu and Mosharaf Chowdhury. Fine-grained gpu sharing primitives for deep learning applications. *Proceedings of Machine Learning and Systems*, 2:98–111, 2020.
- [62] Shulai Zhang et al. Efficient performance-aware GPU sharing with compatibility and isolation through kernel space interception. In *USENIX Annual Technical Conference (ATC)*, 2025.
- [63] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.
- [64] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, and Asaf Cidon. XRP: In-Kernel storage functions with eBPF. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 375–393, Carlsbad, CA, July 2022. USENIX Association.
- [65] Tal Zussman, Ioannis Zarkadas, Jeremy Carin, Andrew Cheng, Hubertus Franke, Jonas Pfefferle, and Asaf Cidon. cache_ext: Customizing the page cache with ebpf. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 462–478, 2025.